

ASSIGNMENT OF SOFTWARE ENGINEERING

1. Software Engineering vs. Traditional Programming

Software Engineering. Is an organized method for creating, maintaining, and operating software. To design software solutions that are dependable, effective, scalable, and satisfy stakeholder needs, it entails adopting engineering concepts and techniques.

Traditional programming. Typically refers to the act of writing code without necessarily following structured methodologies or considering broader software development concerns.

2. Software Development Life Cycle (SDLC)

The software development life cycle is a systematic approach used to develop quality information systems. It determines how an information can support business needs, designing the system, building it and delivering it to users.

Phases of SDLC:

- I. **Planning phase:** This phase is the fundamental process of understanding why an information system should be built, and determining how the project team will go about building it, planning phase activities include project initiation and project management.
- II. **Analysis phase:** The analysis phase answers the questions of who will use the systems, what the system will do and where and when it will be used. The analysis phase activities includes Studying the current systems, requirements gathering, system proposal.
- III. **Design phase:** The design phase decides how the system will operate, in terms of the hardware, software and network infrastructure, the user interface, forms and reports that will be used and the specific programs databases and files that will be needed. The design phase activities include, design strategy, architecture design, database and file specifications and program design.
- IV. **Implementation phase:** During implementation phase the system is either developed or purchased and installed. This phase is usually the longest and most expensive part of the process. The implementation phase include, developing programs, install and test new system, train users and convert to new system.

- V. **System support phase:** Conduct post-implementation system review, identify errors and enhancements and monitor system performance.

3. Agile vs. Waterfall Models

Agile: A group of programming-centric methodologies that focus on streamlining the SDLC. Much of modeling and documentation.

- **Iterative:** Develops software incrementally in short iterations.
- **Flexible:** Emphasizes adaptability to changes in requirements.
- **Collaborative:** Encourages close collaboration between developers and stakeholders.
- Preferred for projects where requirements evolve or need frequent feedback.

Waterfall: Analysts and users proceed sequentially from one phase to the next.

- **Sequential:** Follows a linear process from requirements to deployment.
- **Structured:** Each phase has specific deliverables and review processes.
- **Document-Driven:** Emphasizes documentation and planning upfront.
- Preferred for projects with well-defined and stable requirements.

4. Requirements Engineering

Requirements Engineering is the process of eliciting, documenting, analyzing, validating, and managing requirements throughout the software development lifecycle. It is a crucial early phase in software engineering that focuses on understanding and defining what the software system needs to do to satisfy stakeholders and end-users.

Involves eliciting, documenting, validating, and managing software requirements throughout the SDLC. It ensures that software meets stakeholder needs and is a critical part of reducing project risks and costs associated with requirement changes.

5. Software Design Principles

Modularity in software design involves dividing the software into smaller, manageable parts (modules) that can be developed and maintained independently. It improves maintainability by localizing changes and scalability by facilitating module reuse.

6. Testing in Software Engineering

Levels of Testing:

- **Unit Testing:** Tests individual components or modules.
- **Integration Testing:** Tests interactions between integrated components.
- **System Testing:** Tests the entire system as a whole.
- **Acceptance Testing:** Tests if the software meets user expectations.

Importance: Testing ensures software quality, reduces bugs, validates functionality, and increases user satisfaction.

7. Version Control Systems

Version Control Systems (VCS) track changes to source code over time, allowing collaboration, version management, and rollback to previous states. Examples include Git (distributed) and SVN (centralized).

8. Software Project Management

Role of a Software Project Manager: Manages resources, schedules, risks, and stakeholders to ensure successful project delivery. Key responsibilities include planning, budgeting, communication, and team coordination.

9. Software Maintenance

Software Maintenance involves modifying a software product after delivery to correct faults, improve performance, or adapt to changes in the environment. Types include corrective, adaptive, perfective, and preventive maintenance.

10. Ethical Considerations in Software Engineering

Ethical issues include privacy violations, biased algorithms, and intellectual property theft. Engineers should adhere to ethical codes, consider societal impacts, and advocate for responsible use of technology.

These topics provide a comprehensive overview of key aspects in software engineering, from development methodologies to ethical considerations, essential for understanding the field's breadth and depth.